

琥珀

会自己动手把事情办完的私人助理 · 作品设计说明

你对着它说一句，它真的去做，做完回来告诉你。

像豆包那样陪你聊天，像 Codex 那样写程序；但它更是"你的"：专属记忆、专属工具，还能给你造你想要的小程序，帮你解决生活里的问题。

参赛作者：_____ 年级：_____

学校：_____ 指导教师：_____

完成日期：2026 年 ____ 月 ____ 日 演示方式：现场真机演示（手机 / 电脑浏览器）

线上跑到的东西：<https://w.stalkeraid.cn> | 这份说明：<https://w.stalkeraid.cn/hupo-design-note.pdf>

第一部分 它是什么，为什么值得做

琥珀是一个会自己动手把事情办完的私人助理：你对着它说一句，它真的去做，做完回来告诉你。

它能像豆包那样陪你聊天，也能像 Codex 那样写程序；但它更是“你的”：专属记忆（关掉、重启、换设备都还记得）、专属工具（在你自己的机器上替你办事），还能给你造你想要的小程序，帮你解决生活里的问题。它是一个常伴你左右的小助手。

给谁用：会说话的人都能用

八九岁的孩子（还没学打字）· 六七十岁的老人（不用学拼音）· 在手机、平板上移动办公的人。它把“输入”这一关从键盘换成说话。

为什么值得做：卡住人的往往不是想法，是“输入”

这两年我用 AI 的方式变了：以前要坐到电脑前，现在越来越多是在手机上、平板上把事办了。可越是这样越发现一件事：**我说得比打得快得多**。想清楚一件事只要一瞬，把它一个字一个字敲进输入框却要花很久，还得先会打字、打准、打对。而说话是人天生就会的：**孩子先会说话，很多年后才学会写字；老人打不了拼音，但话说得很清楚**。

⇒ 所以这件作品要做的不是“又一个聊天软件”，而是**让“说话”就够了**：你说，它去做；做完，它回来说给你听。

它能做什么

- **说话就行**：打字、按住麦克风说话，都能用。
- **真去办事**：查资料、算数、写代码、画图、记账、上网查东西。
- **会造软件**：说一句「帮我做一个记账的小程序」，桌面上**当场长出一格**，点开就能用，数据还存得住。
- **记得住**：关掉页面、甚至机器重启、换一台设备，它还记得之前说过、做过、答应过的事。
- **做完回来说**：长一点的活它先放后台，办完或失败都主动说一声；随时可以问「那件怎么样了」。
- **看得见**：它正在做什么、用过哪几样工具、花了多少，都摆在聊天记录里，不是黑箱里冒出一个答案。
- **听不清会问**：说得含糊时，它先用一个只负责“听懂”的快速模型问一句（**语音 + 文字**），问明白才发出去，不替我乱猜。

它跟常见的 AI 助手不一样在哪

（跟豆包、元宝、Codex 这一类比，主要区别在**第二部分 1.5**：它们是云上的 App，替你说话、替你在手机上点；琥珀长在你自己那台机器上，替你办事，还给你造软件。）

常见的毛病	琥珀怎么处理
只会说，不会做	它手里有真的工具，而且 每一轮活都落账 ：做了什么、成没成、花了多少，都查得到
做完不说，用户不知道办没办	收口时主动说结果；失败也主动说清是哪一步没成，不悄悄卡住
中途被打断就"失忆"，或假装没发生	还开着的活逐件记在盘上；重启后逐件如实回答"做完了 / 已停"，不会变成"查无此事"
要会打字才能用	会说话就能用（这一版就是从这里开始的）
不敢把隐私交给它	一人一台隔离的小主机 ：他的数据、他的钥匙、他的助手都在自己那台里



图1 真实运行中的桌面（线上截图·取自一个**真实用户账号**，不是开发者的演示环境）。每一格是一个小程序，也就是一条对话；左下角「+」是"造一个新的"。底图那张壁纸是**这个账号自己挑的**——它跟着账号走，换一台设备登录看到的还是它。

第二部分 技术

1 技术原理

1.1 总体结构

系统由三部分组成：用户手上的**界面**、一台负责转发与记账的**中心服务**，以及**每个人自己的那台小主机**（用容器做的隔离环境，我们叫它"盒子"）。真正干活的东西（助手、聊天记录、小程序、钥匙）全都在用户自己的盒子里。

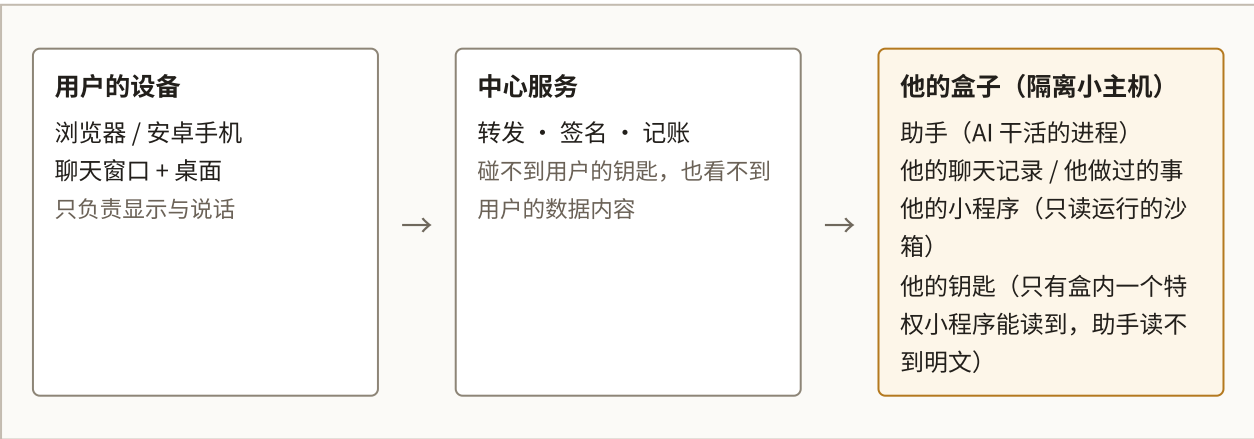


图2 系统结构：一人一台盒子，盒子之间互相看不见。

1.2 一轮对话：一条有始有终的流水线

用户说一句话，在系统里不是"丢给大模型答一句"，而是变成**一件有始有终、还落了账的活**：

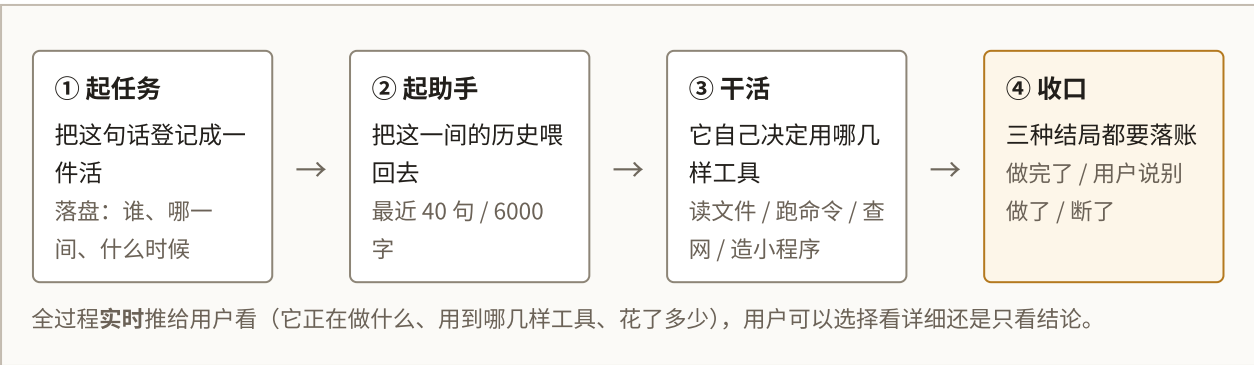


图3 一轮对话的流水线。"有始有终"是整个系统的地基：没有收口的活，重启后要能逐件如实交代。

1.3 人工智能用在哪三处

(1) 当"大脑和手"：由大模型驱动的工具调用

助手背后是一个大语言模型（低推理开销那一档，响应快）。它做的不是"续写文字"，而是**决定下一步用哪样工具、看结果、再决定下一步**，直到这件事办完。这些工具的调用与结果都进聊天窗口的记录里。也就是说，**用户能看见 AI 是怎么一步步做事的**，不是黑箱里冒出一个答案。

(2) 当"程序员"：由 AI 写出来的小程序，跑在沙箱里

这是本作品最有意思的一段。用户说「帮我做一个记账的小程序」，系统做了四件事：

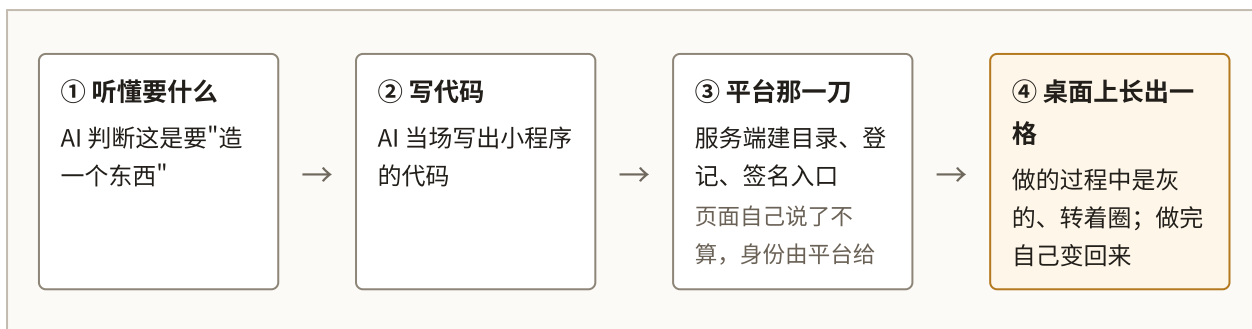


图4 一个小程序从"一句话"变成"桌面上的一格"。

每个小程序都跑在一个关起来的沙箱里：代码是**只读**挂载进去的；每个小程序有自己的一格数据库，**别的程序读不到**；它想联网、想连哪个网站、想用 AI，都要用户**第一次打开时点过头**才生效（默认什么都没有）。它上网还会留痕：去过哪些域名、多少次，都记在账上（不记录内容）。

(3) 当"耳朵和嘴"：听得见，还要听得懂

现在已经做到的：按住麦克风说话，语音**流式识别**成文字（用的是豆包大模型语音识别）。而**听清楚了，不等于理解对了**。所以中间加了一层**只听懂、不创作**的快速模型：

- 它把用户说的话理顺（补标点、改同音错字），**但不许添加用户没提过的要求**；
- 它只回答"我哪里不确定"（是上周还是上个月？给谁看？要不要存数据？）；
- 不确定就**问回去**（同时用语音念出来 + 文字流式显示），问明白了才自动发出去；
- 问了两轮还不清楚，就不硬发，屏幕上如实说"这句我没听清，你再说一遍"。

这一层的关键设计是"**宁可问一句，也不替用户猜**"：因为猜错以后，助手会真的照错的去做。它跟场景分工也一样：要小程序的时候，允许它多问一两句（要不要存数据、要不要提醒），但每一句都要用户答过才算数。

1.5 它跟豆包 / 元宝 / Codex 这类助手，主要区别在哪

先说清楚**不比什么**：琥珀用的就是通用大模型的接口，**自己并不训练模型**（语音识别用的也是豆包那一套）。论回答质量、生态规模、语音成熟度，我们比不了这几家。下面比的不是"谁更聪明"，而是"**形状**"不一样。

比什么	琥珀	豆包 / 元宝（通用 AI 助手 App）	Codex / Claude Code（给程序员用的编码智能体）
干活的地方	你自己那台小主机里（他的文件、数据、钥匙都在那儿）	厂商的云上；要动你的手机或文件，得靠系统级授权与插件	你的 代码仓库 里（云端沙箱或本机终端）
交出来的是什么	桌面上多出一格能用的小程序 （当场写、点开就能用、数据存得住）+ 事情真的办完	回答 / 总结 / 写文档 / 搜资料；手机助手那一支能在多个 App 之间替用户点（比价下单、查票订票那一类）	可以合并的 代码改动 （补丁 / PR）+ 测试结果
主要怎么	语音优先 ：说一句就行（听不清会问一句再发）	打字为主；语音很成熟（豆包有语音 / 视频通话、耳机唤醒）	打字为主（命令行 / IDE / 聊天侧边栏）

输入			
记忆放在哪	记在他自己的盒子里：关掉、重启、换设备都还在；删号即删盒	记在厂商账号里	记在项目／仓库上下文里
面向谁	会说话的人都能用：八九岁的孩子到六七十岁的老人	所有普通用户（这一层它们做得比我们大得多、也快得多）	开发者（写代码这件事上专业得多）
安全边界能不能查	助手只能提申请、人按按钮・钥匙不出盒子・小程序权限弹窗・开机完整性清单—— 每一条都在仓库里可查、有判据盯着	厂商自证（用户看不到边界是怎么实现的）	权限交给开发者自己管（沙箱／审批）
能不能改	整个实现是我们自己写的（183 篇实现契约 + 1525 条判据），每一处都能动	封闭的 App	面向专业开发者，改的是代码不是"助手本身"

一句话的区别：它们把 AI 放在自己的云上，替用户说话；琥珀把 AI 放到用户自己那台机器上，替用户办事，还给用户造软件。

⇒ 由此长出三件它们今天不是这个形状的事：① 给普通用户"造一个只属于他的小程序"，做完就留在他的桌面上；② 每一件活的来龙去脉（做过什么、成没成、花了多少）在用户自己的机器上逐条留账，删号就一起走；③ 钥匙和数据不出他那一台：助手连"读钥匙"这条路都没有。

（对比依据是各家公开发布的介绍与报道，资料截至 2026-10；这里面豆包"手机助手"那一支还是技术预览版，官方也说明"受限于大模型的不确定性，相关场景无法保证百分百复现"。这一节只说明形状不同，不代表谁好谁坏。我们自己并不训模型，用的是通用大模型的接口。）

1.4 怎么做到"你的东西只有你有"

- 一人一台盒子：每个用户一个独立容器（无特权容器 + 去掉全部特权能力后再按需加回），代码层只读挂载，每台限制内存（现在每台 4 GiB）；一台出问题不影响别人。
- 钥匙不出盒子：模型钥匙放在盒子里的一个特权小程序手里，干活的助手环境里只有一个占位符，它连"读钥匙"这条路都没有。
- 删号即删盒：账号不要了，那台盒子一起删掉。

2 实现过程

我用了一套"先立规矩、再写代码，而且每一步都要能被推翻"的做法：

1. 先写"要什么、为什么"：把设计决定一条条写下来（为什么这么做、代价是什么、以后不许怎么做），前后做了两轮独立评审。这些文档是"规矩"，不是"事后说明"。
2. 判据先行：每一件功能都配"怎么算做完了"的自动判据；代码写完，判据必须通过。
3. 负向对照 + 变异验证：光"通过"不算。每条判据都要证明它抓得住错：我会故意把那条规矩删掉或改坏，看判据是不是当场变红；不红就说明这条判据是空转的。

4. **每一步留账**：做了一批就记一行账（做了什么、读数是多少、还欠什么）。现在仓库里有 **183 篇实现契约**，全部可追溯。
5. **边做边真上线**：目前版本 **V1.0**（2026-10-04 定档），已经部署在真实的服务器上、真机上在用（现在有三台用户的小主机在跑）；之后只修严重问题，新功能进 V2.0。

这一段最想说明的是：**这个作品不是"演示原型"**。它有真用户、真服务器、真容器，也有真出过的故障和修法（例如有一次浏览器端因为地址算法写错导致数据流被浏览器拦掉，而当时所有自动判据都是绿的，因为判据绕开了出错的那一行。我们把这条教训写成了新规矩：**凡是客户端自己算出来的东西，检查就必须打在客户端这一侧**）。

3 测试结果

检查	结果（2026-10-04 实测）
服务端自动判据	1525 条全部通过，0 条失败 （分布在 142 个判据文件里）；其中不少是端到端判据：真起服务、真连连接、真开容器
客户端自动判据	静态分析 0 问题 ；逻辑判据全过； 可访问性硬检查 （3 倍字号五档不破版 + 可点范围 ≥ 44 + 不封顶）全过；其余 130 份 界面判据全过
"线上跑的就是代码这一版"	漂移检查：网页产物指纹与仓库一致；三台用户盒子跑的程序版本与仓库同一份
真机（公网）	对外服务 200；浏览器探针实测：页面打得开、 它自己建立的数据连接通 （1 条连接、202 帧、登录凭证自动续期）；并人工看了截图确认界面正常
文档检查	203 份文档里的相互引用 全部指向存在的文件 （没有悬空引用）
开机完整性	222 个"开机必读"文件有哈希清单（root 只读）： 被改过一个字节就拒绝启动

怎么读这些数字：**1525** 不是"跑了 1525 次断言"，而是 1525 条"这件事应该是什么样"的判据；而且每条判据都要求能被"改坏代码"证明它是有效的（第 2 节的第 3 条）。

4 安全考虑

#	防线	具体做法
1	助手只能提出申请，签字的是人	要改动运行中的系统（装东西、改配置），助手把命令准备好，由人按键执行。理由是：真正的危险不是"它干一次坏事"，而是"坏事能不能留下来"——比如它读到一段看不见的网页内容，被诱导写进自己的配置，从此每次开机都读它，而表面上一切正常
2	一人一台盒子	容器隔离 + 只读代码层 + 资源上限；跨用户看不到对方任何东西
3	钥匙不出盒子	盒内特权小程序握钥匙，助手只有占位符；日志和回复里永不出现钥匙

4	小程序：声明 ≠ 已给	小程序说"我想要联网 / 想用 AI"只是它想要；用户第一次打开时一张弹窗一次问完，点过头的才生效；数据天生就有存储，但每格独立、跨格读不到、随时能一键清空
5	上网留痕	助手能访问网络，但去过哪、多少次都落账（不记录内容），可查
6	开机完整性清单	决定"它下次开机读什么"的文件（人格、规范、能力清单），哈希写在 root 只读清单里，动过就拒绝启动；每次重建都留痕
7	删除要二次确认	删之前先把它要带走的東西（聊天 / 数据 / 经验）逐条摆出来再问一次；服务端是软删除，还能拿回来
8	页面不许说假话	做不到的事不许画按钮；拿不到的事实不许猜（"不知道"就说不知道）。这条被当成缺陷来管，有判据盯着

5 创新点与自我评价

评委关注点	本作品对应的东西
技术性	自己实现的调度内核（可收口的轮次、落盘账本、崩溃后接管）· 自建"一人一台盒子"的容器编排 · 流式语音识别 · 小程序沙箱（只读挂载 + 逐格数据库 + 权限弹窗 + 上网留痕）· 1525 条判据 + 变异验证 的工程方法
智能性	大模型不只是聊天：它会 写代码造小程序 、长期记住用户的事、把长活放到后台自己办完再汇报；还加了"先说懂、再问一句、才发出去"的理解层
实用性	使用者很宽：从八九岁的孩子到六七十岁的老人，到在手机/平板上移动办公的人 —— 会说话就能用 ；一句话就能办事；桌面上的东西看得见、摸得着、能撤；已经在真实环境运行，不是演示品
安全性	第四节的八条防线；其中最硬的三条是"助手只能提申请、人按按钮"、"钥匙不出盒子"、"开机完整性清单"

6 不足与改进方向

- **语音输出还没接通手机**：网页上已经能用设备自带的声音念出来，手机（安卓包）还不能；正在接通云端语音合成。
- **登录还靠临时验证码**：正式短信通道还没接，这是"已知并接受"的临时状态，不是最终形态。
- **性能还没优化**：目前策略是"先功能、后性能"，所以内存占用、并发上限都还比较宽松。
- **"助手卡住"还有一类情况不能自动收尾**：进程死掉能自动收场，但"进程活着却卡在不动的循环里"目前没有自动收尾（这是为了让长活不被时间上限砍掉而做的取舍，写在设计记录里）。
- **平台只有网页和安卓**：iOS 还没做。

附录 A 现场演示步骤（约 4 分钟）

时间	演示什么	怎么讲
0:00	打开页面 → 登录 → 桌面	"这是他自己的桌面。每一格是一个小程序，也是它的一条对话。"
0:30	按住麦克风说一句人话（语音输入）	"我不用打字，说一句就行。"（指屏幕上的字）
1:00	让它做一个小程序："帮我做一个记账的"	"它现在真的在写代码。"（指桌面上新长出来的那一格：灰的、转着圈）
1:45	等它做完，那一格自己变回正常颜色，点开用一下	"做好了它自己变回来，点开就能用。"（记一笔，再打开还是那笔 —— 数据存住了）
2:45	让它干一件长一点的活，然后合上页面	"长活它自己放后台办，办完会主动告诉我。"
3:15	回到桌面，看"在做"的小圈在转	"哪一件还在办，桌面上看得见。"
3:30	说一句含糊的话，演示"它问一句"	"听不清它会问，问清楚才发出去 —— 不替我乱猜。"
3:50	打开设置里的安全项目	"他自己的钥匙、他点头才开的权限、还有这份开机清单。"

演示提示：现场演示前先进入「设置」里的演练入口。那里是一个空白、临时的对话系统，整条流程真跑，但一句都不会真的发出去，所以可以反复演示，不怕说错、不留垃圾。

附录 B 名词解释（给非技术评委）

词	在这个作品里是什么意思
大模型 / 大语言模型	能理解并生成自然语言的 AI 模型。这里它不只用来聊天，主要用来"决定下一步做什么、用什么工具"。
容器（盒子）	一种轻量的隔离技术：在同一个操作系统里划出一块独立的小空间，里面有自己的一套程序和文件，互相看不见。这里"一人一台盒子"。
语音识别 / 语音合成	前者把说的话变成文字（耳朵），后者把文字念出来（嘴）。
判据 / 负向对照 / 变异验证	判据 = 一条"这件事应该是什么样"的自动检查；负向对照 = 先证明这个检查抓得住错误；变异验证 = 故意把规矩改坏，看检查是不是当场变红。
沙箱	把一段程序关起来运行的环境，限制它能碰到什么（这里是：代码只读、数据库一格独立、联网要用户点头）。